

UNIVERSITÀ DEGLI STUDI DI MILANO
FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA
GIOVANNI DEGLI ANTONI



MIKROMOOD
SINTETIZZATORE AUDIO BASATO SU STM32

Docenti: Bruschi Danilo Mauro
Pedersini Federico

Proposta di:
Bianchi Davide
12820A

ANNO ACCADEMICO 2023-2024

Indice

1	Introduzione	1
1.1	Obiettivi del progetto	1
1.2	Utilità del progetto	2
1.3	Struttura del sintetizzatore	3
1.4	Protocolli	4
1.4.1	USB	4
1.4.2	MIDI	4
1.4.3	I ² C	5
1.4.4	I ² S	5
1.5	Il convertitore digitale-analogico CS43L22	5
2	Architettura del Sistema	6
2.1	Architettura Generale Completa	6
2.2	Architettura Software Completa	8
2.3	Gestione degli input	9
2.3.1	Digitali	9
2.3.2	Analogici	10
2.4	Gestione del MIDI	10
2.5	Gestione del DAC	12
2.6	Gestione dell'algoritmo del synth	13
3	Componenti Hardware e Circuiteria	13
3.1	Circuiteria degli input	15
3.2	Circuiteria degli output	15
4	Analisi dei Tempi di Esecuzione	16
5	Analisi di Potenza Dissipata	17
6	Conclusione	17

1 Introduzione

1.1 Obiettivi del progetto

L'obiettivo di questo progetto è sviluppare un sintetizzatore audio digitale chiamato MikroMOOD ispirato dal Minimoog Model D, uno dei sintetizzatori analogici più iconici

della storia, creato dalla compagnia americana Moog Music nel 1970.

In particolare, verrà sviluppato un prototipo semplificato di tale sintetizzatore che ri-specchi le caratteristiche sonore principali e che ovviamente possa essere utilizzato come un qualunque strumento musicale. Le tematiche principali affrontate sono le seguenti:

- Gestione degli input analogici e digitali del pannello di controllo usando la HAL¹ fornita da ST.
- Gestione del DAC CS43L22 presente sulla board. L'obiettivo è quello di implementare un driver per comandare il convertitore D/A tramite il protocollo I^2C e scambiare i dati audio tramite il protocollo I^2S .
- Sviluppo e implementazione del protocollo USB-MIDI per interfacciarsi con la tastiera. Questo implica lo studio della USB Host Library fornita da ST come punto di partenza per poter sviluppare le parti necessarie per la gestione del protocollo MIDI 1.0.
- Sviluppo e implementazione degli algoritmi di generazione ed elaborazione dei segnali audio.
- Montaggio e cablaggio del circuito del pannello di controllo su breadboard.

La sfida è quella di riuscire a creare uno strumento real-time scrivendo algoritmi ottimizzati per dispositivi con poco spazio in memoria e poca potenza computazionale.

1.2 Utilità del progetto

Di seguito alcuni punti in cui il progetto può risultare utile:

- Flessibilità e Versatilità: Il synth, sebbene sia una versione compatta, consente agli utenti di esplorare e creare una varietà di suoni unici e personalizzati.
- Apprendimento ed Esplorazione: Il progetto fornisce un'opportunità per gli studenti, gli appassionati di musica e gli sviluppatori interessati alla sintesi audio digitale. Attraverso lo studio e l'implementazione pratica di algoritmi di sintesi, gli utenti possono approfondire la propria comprensione della teoria musicale e dell'elaborazione del segnale digitale.

¹HAL: Hardware Abstraction Layer

- **Integrazione e Personalizzazione:** Grazie alla flessibilità offerta dal microcontrollore STM32, il sintetizzatore può essere integrato in una vasta gamma di applicazioni musicali e audiovisive. Per esempio può essere utilizzato sia come strumento musicale che per la generazione di effetti sonori o scenografici.
- **Innovazione e Sperimentazione:** Il progetto sviluppato è solo un punto di partenza. Esso incoraggia l'innovazione e la sperimentazione nell'ambito della sintesi audio digitale. Gli sviluppatori di algoritmi audio possono utilizzare la struttura programmata per aggiungere nuovi algoritmi che possono creare effetti particolari ed adattarli alle loro esigenze.

1.3 Struttura del sintetizzatore

La figura 1 mostra più in dettaglio il pannello di controllo del sintetizzatore usato per modificare il suono tramite gli appositi interruttori (colorati in nero) e manopole (colorate in bianco e grigio). Si descrivono brevemente le varie sezioni:

- **Oscillatori:** sono la sorgente del suono. I loro parametri possono essere modificati tramite le rispettive manopole:
 - *Waveform* modifica la forma d'onda.
 - *Range* modifica l'ottava di riferimento.
 - *Frequency* modifica la frequenza.
 - *Volume* modifica il volume.
 - *On/Off Switch* accende/spegne l'oscillatore.
- **Filtro:** permette di modificare lo spettro del segnale prodotto dagli oscillatori.
- **ADSR:** permette di modificare nel tempo il volume del sintetizzatore.
- **Modulazioni:** permettono di modificare il suono prodotto dagli oscillatori usando come modulante un LFO (Low Frequency Oscillator).
In questo prototipo ne sono implementate 3: modulazione della frequenza di taglio del filtro, modulazione dell'ampiezza (tremolo) e della frequenza (vibrato) degli oscillatori.

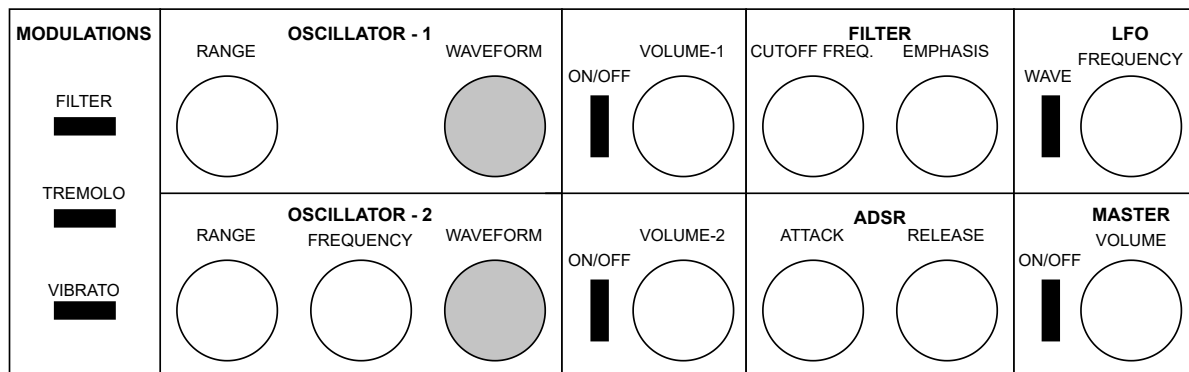


Figura 1: Pannello di controllo del MikroMOOD.

1.4 Protocolli

1.4.1 USB

Il protocollo USB (Universal Serial Bus) è uno standard di comunicazione utilizzato per collegare tra loro dispositivi elettronici e computer. Questo standard è stato introdotto negli anni '90 per semplificare e migliorare la connettività tra i dispositivi, eliminando la necessità di numerosi tipi di connettori e porte. L'USB è un'interfaccia seriale con le seguenti caratteristiche: semplicità e flessibilità (plug-and-play), bidirezionalità, aumento della velocità, basso costo. In questo progetto è utilizzato per la comunicazione tra microcontrollore e tastiera MIDI.

1.4.2 MIDI

Il protocollo MIDI (Musical Instrument Digital Interface) è uno standard di comunicazione utilizzato per consentire a strumenti musicali elettronici, apparecchiature audio e software musicale di comunicare tra loro. Esso trasmette informazioni serialmente relative alla musica, come note, velocità, espressione e altri parametri di controllo, permettendo di controllare strumenti musicali elettronici, sintetizzatori, workstation audio digitali (DAW), software musicale e altro ancora. Il protocollo MIDI è stato sviluppato negli anni '80 diventato uno standard ampiamente utilizzato nell'industria musicale e continua ad essere fondamentale nella produzione musicale contemporanea. In questo progetto è utilizzato per lo scambio di messaggi musicali dalla tastiera al microcontrollore.

1.4.3 I²C

Il protocollo I²C (Inter-Integrated Circuit) è un protocollo di comunicazione seriale a due fili sviluppato da Philips (ora NXP Semiconductors) negli anni '80. È utilizzato per consentire la comunicazione tra microcontrollori, sensori, dispositivi di memoria, convertitori di dati e molti altri dispositivi elettronici. Il protocollo I²C utilizza due linee di segnale, SDA (Serial Data Line) e SCL (Serial Clock Line), per trasmettere dati e segnali di clock, rispettivamente. Queste linee consentono la comunicazione bidirezionale tra un dispositivo master e uno o più dispositivi slave collegati nello stesso bus. Il master inizializza e controlla tutte le comunicazioni sul bus, mentre gli slave rispondono alle richieste del master. In questo progetto è utilizzato per poter comunicare con il convertitore digitale-analogico presente sulla board e inizializzare i suoi parametri,

1.4.4 I²S

Il protocollo I²S (Inter-IC Sound) è un protocollo di comunicazione seriale utilizzato principalmente per trasferire dati audio digitali tra componenti elettronici come microcontrollori, processori audio digitali, codec audio, convertitori analogico-digitali, convertitori digitale-analogici e altri dispositivi audio. A differenza di altri protocolli come l'I²C che sono più adatti per la trasmissione di dati di controllo e configurazione, l'I²S è specificamente progettato per la trasmissione di dati audio digitale ad alta fedeltà. Esso trasmette segnali audio digitali sotto forma di flussi di dati sincronizzati, compresi i dati audio e i segnali di clock correlati. In questo progetto viene utilizzato per inviare i campioni del suono generato al convertitore digitale-analogico presente sulla board.

1.5 Il convertitore digitale-analogico CS43L22

Il CS43L22 è DAC stereo a bassa potenza dotato di amplificatori per gli auricolari e per gli speaker. Uno schema generale del dispositivo è mostrato in figura 2. Come si può notare, a sinistra è presente una sezione digitale che si occupa della comunicazione con il microcontrollore. Attraverso il protocollo I²C è possibile inizializzare il dispositivo scrivendo nei suoi registri i valori necessari usando la I²C Control Port. Attraverso il protocollo I²S è possibile inviare i dati audio digitali dal micro al dispositivo. Man mano che i bit sono inviati al chip vengono elaborati da un Digital Signal Processor e poi convertiti da un DAC stereo delta-sigma a 24 bit (nominali) per poi essere amplificati e portati alle uscite left e right delle cuffie. C'è anche la possibilità di collegare direttamente 2 speaker grazie alla presenza di 2 amplificatori in classe D. In questo caso lo stream di bit che rappresenta il segnale audio viene modulato tramite un modulatore PWM, amplificato

e portato in uscita. Sono inoltre presenti 4 input stereo (Left/Right inputs) analogici che possono essere sommati e combinati tra di loro per poi essere portati all'uscita degli auricolari.

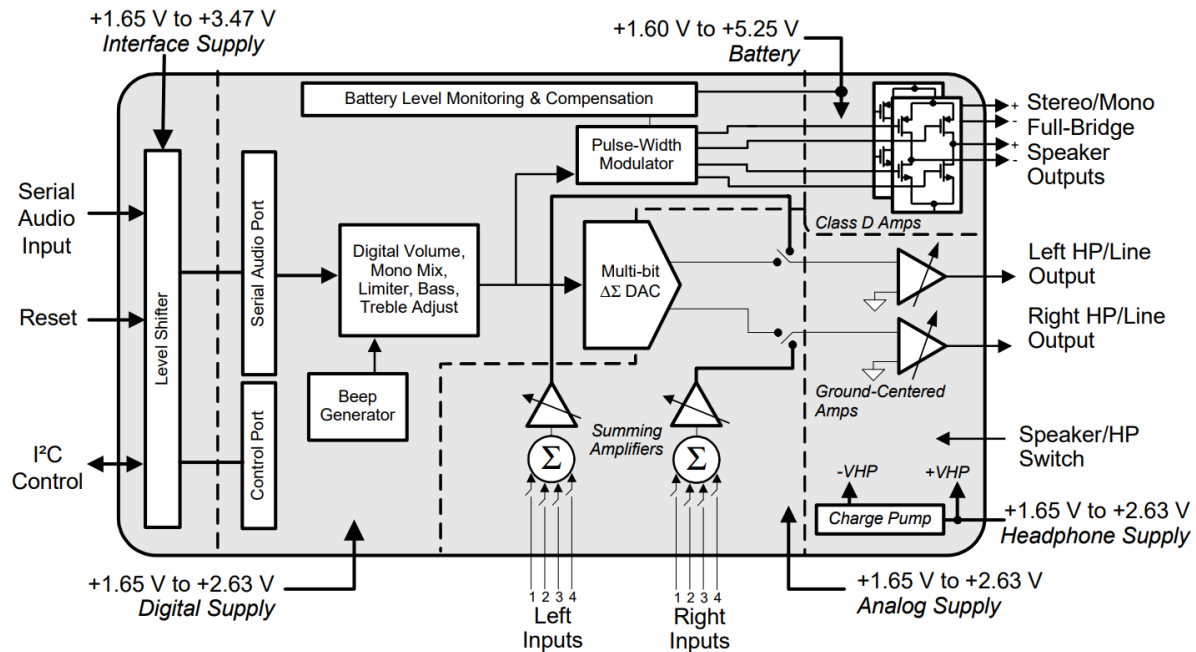


Figura 2: Schema a blocchi del CS43L22.

2 Architettura del Sistema

2.1 Architettura Generale Completa

Nel modo più semplice in assoluto, il sintetizzatore può essere visto come un dispositivo che, in base ai segnali che riceve in input dalla tastiera e dal pannello di controllo, fornisce in output un segnale audio che viene amplificato ed emesso dagli speaker (si veda la figura 3).

La figura 4 mostra invece uno schema a blocchi della struttura interna del sintetizzatore. In questo caso viene mostrata la catena di elaborazione del segnale evidenziando i singoli blocchi e i parametri dell'interfaccia che li modificano. In sequenza abbiamo:

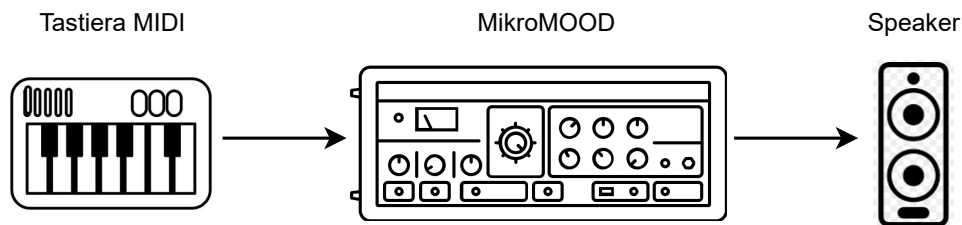


Figura 3: Schema generale del sintetizzatore.

- Calcolo delle modulazioni nel blocco Osc.Modulation. Esso prende in input la nota suonata dalla tastiera, il pitch wheel e la modulazione introdotta dal LFO (se attiva) creando un effetto di vibrato.
- Generazione delle forme d'onda tramite gli oscillatori 1 e 2. I parametri modificabili sono: la forma d'onda, l'ottava, il detune e la modulazione in frequenza precedentemente calcolata.
- Gestione del volume, accensione e spegnimento degli oscillatori tramite il Mixer.
- Filtraggio del segnale tramite il filtro Ladder. I parametri modificabili sono la frequenza di taglio, la risonanza e la modulazione della frequenza di taglio (se attiva).
- Guadagno finale e ADSR. I parametri modificabili sono l'attack e la release, inoltre è possibile modulare il guadagno creando un effetto di tremolo.

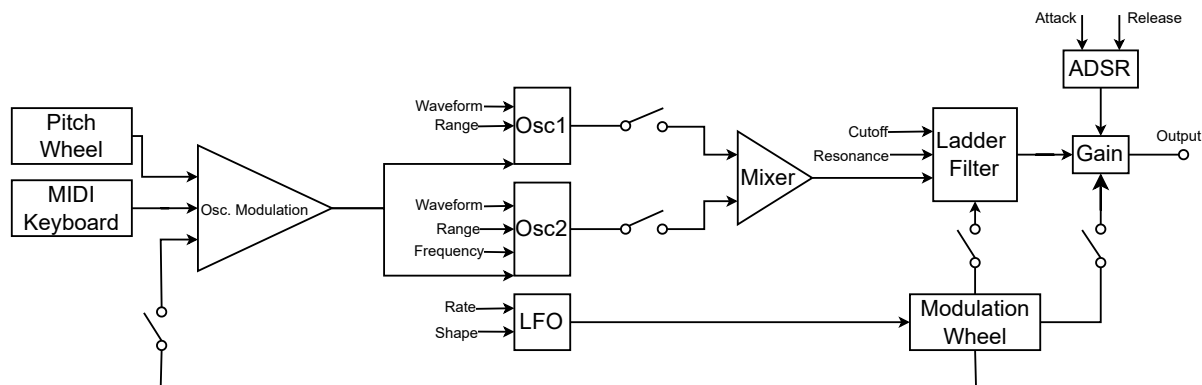


Figura 4: Schema a blocchi del sintetizzatore.

2.2 Architettura Software Completa

A livello software, l'intero progetto si compone dello sviluppo di 5 macroblocchi:

- Gestione degli input digitali; descritto nel paragrafo 2.3.1.
- Gestioni degli input analogici; descritto nel paragrafo 2.3.2.
- Gestione dell'input USB_MIDI; descritto nel paragrafo 2.4.
- Gestione dell'output audio al convertitore digitale-analogico; descritto nel paragrafo 2.5.
- Gestione degli algoritmi di generazione del suono; descritto nel paragrafo 2.6.

Una delle proprietà del sistema è che deve essere real-time, questo implica che la modifica di un qualsiasi parametro deve essere letta prontamente. Per questo motivo il software sarà diviso in una parte a bassa priorità ed una ad alta priorità come mostrato in figura 5. Considerando i 5 macroblocchi presentati sopra si può dire che:

- Gli input digitali e analogici vengono gestiti rispettivamente tramite interrupt e DMA.
- La ricezione dei messaggi dalla tastiera MIDI viene invece fatta nel ciclo while. Il microcontrollore si mette in ascolto sul canale attendendo un interrupt che gli notifichi l'arrivo di un messaggio. il sistema non è stato gestito interamente via interrupt (senza stare in ascolto sul canale nel ciclo while) perchè il middleware fornito da ST impone questa struttura del programma.
- L'invio dei campioni audio alla periferica I2S avviene tramite DMA.
- Il processing dei campioni audio viene fatto nella callback del DMA in modo che non venga bloccato da altri eventi. Si è provato a spostare il rprocessing nel ciclo while (a bassa priorità) ma i suoni erano distorti probabilmente a causa della gestione del protocollo USB.

Tutti gli interrupt hanno la stessa priorità e questo non comporta ritardi o conflitti nell'utilizzo del prototipo.

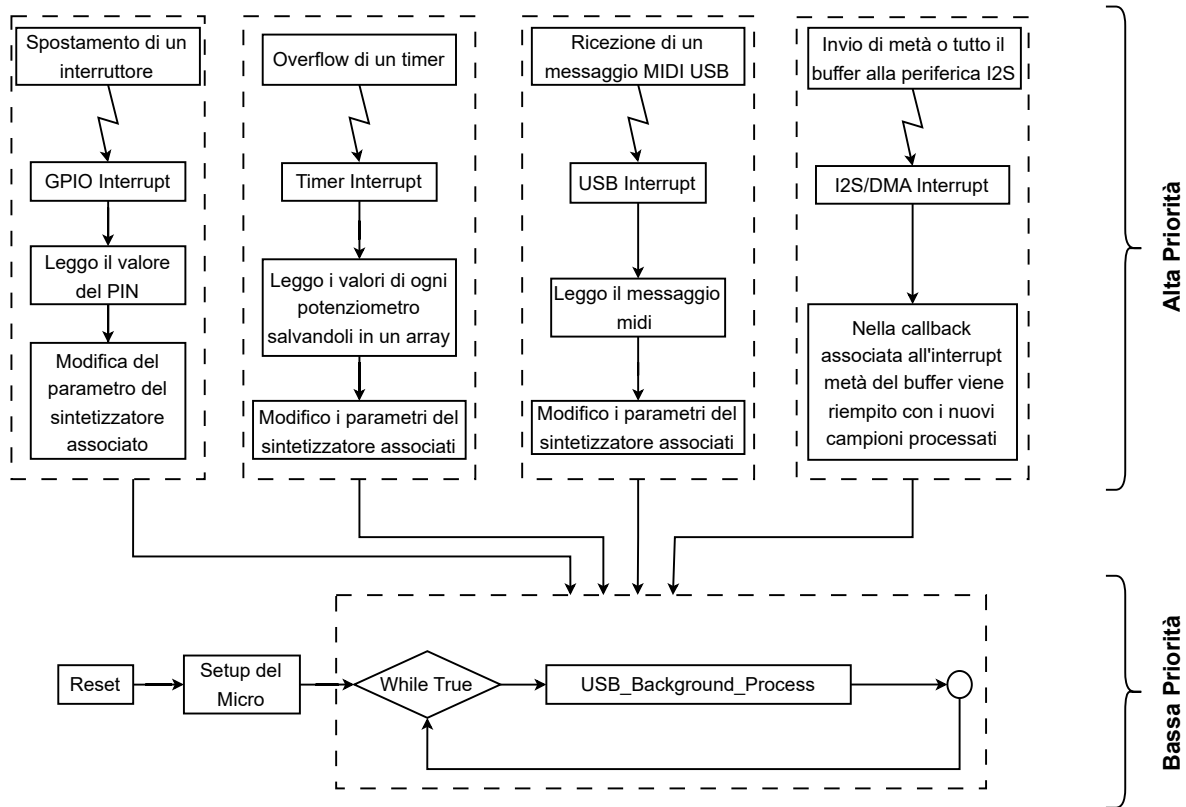


Figura 5: Diagramma software completo.

2.3 Gestione degli input

2.3.1 Digitali

I GPIO (General Purpose Input/Output) sono pin che possono essere configurati per funzionare come ingressi o uscite digitali, consentendo al microcontrollore di comunicare con il mondo esterno. Come mostra la figura 6, i GPIO sono stati inizializzati in modalità interrupt, in particolare in modalità 'GPIO_MODE_IT_RISING_FALLING', per poter rilevare il cambio di stato del pin da basso a alto (rising edge) e da alto a basso (falling edge) dopo aver spostato l'interruttore. Quando viene rilevato un interrupt su un GPIO, il microcontrollore esegue una routine di gestione degli interrupt nel quale viene letto lo stato del pin modificato per poi cambiare i parametri del sintetizzatore.

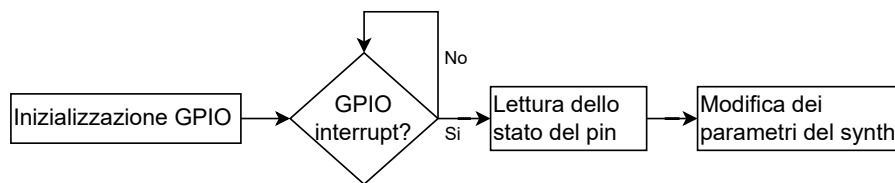


Figura 6: Diagramma di flusso dei GPIO.

2.3.2 Analogici

L'ADC (Analog to Digital Converter) è un dispositivo in grado di convertire una grandezza analogica in un valore digitale. Come mostrato nel diagramma in figura 7, nella fase di inizializzazione viene: abilitato lo stream0 e l'IRQ (Interrupt Request) associato al DMA, impostato il timer 2 affinché la frequenza delle conversioni sia di 5 Hz ($T = 200\text{ms}$), impostato l'ADC1 affinché la risoluzione sia di 12 bit, il numero di canali siano 11 e la conversione venga abilitata dal fronte di salita del timer (Scan Conversion Mode). Nella fase di avvio viene abilitato il timer2 e viene avviata la conversione degli ADC usando il DMA per il trasferimento dei dati dalla periferica alla memoria senza l'uso della CPU. Lavorando in questa modalità il convertitore, ad ogni fronte di salita del timer2 (200 ms), converte sequenzialmente gli ingressi associati, il DMA (in modalità *circular*) si occupa di trasferirli in memoria e poi viene chiamata una callback per notificare la CPU che i nuovi dati sono pronti per essere utilizzati e modificare i parametri del sintetizzatore.

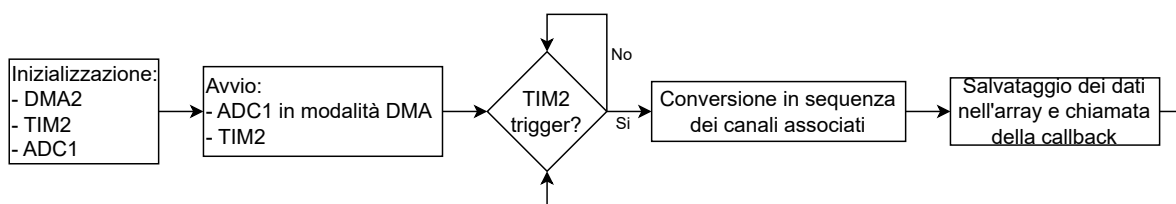


Figura 7: Diagramma di flusso dell'ADC.

2.4 Gestione del MIDI

I messaggi MIDI sono inviati dalla tastiera (device) al microcontrollore (host) tramite il protocollo USB. ST mette a disposizione un middleware per la gestione dell'USB, ma

per la gestione del MIDI è stato necessario scrivere un modulo apposito. Nella prima fase (eseguita nella parte di programma a bassa priorità) mostrata in figura 8, viene inizializzato il protocollo definendo il modulo che gestisce l'USB e la velocità della comunicazione, resettando tutte le configurazioni precedenti e gli endpoint, impostando i parametri del driver a basso livello collegandoli all'handle. Successivamente viene collegata la classe MIDI_USB al dispositivo (microcontrollore) in modo che esso possa gestire la comunicazione con la tastiera. Infine vengono avviati i protocolli e i driver a basso livello.

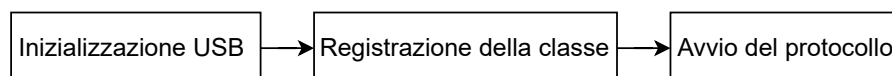


Figura 8: Fasi preparatorie del protocollo USB.

Nella seconda fase (anch'essa eseguita nella parte di programma a bassa priorità) mostrata in figura 9 mi occupo di controllare che il dispositivo sia connesso correttamente, della sua enumerazione (imposto l'indirizzo del dispositivo ed ottengo alcuni suoi descrittori sul tipo di device e sulle configurazioni supportate) e della selezione di una delle configurazioni del dispositivo connesso. Successivamente controllo che l'USB host (microcontrollore) supporti la classe del dispositivo device (tastiera MIDI) e in caso positivo inizializzo la classe per la sua gestione.

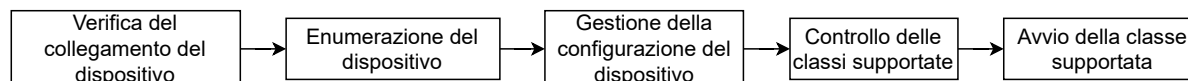


Figura 9: Fase di riconoscimento del dispositivo USB.

Nella terza fase mostrata in figura 10 viene descritto il funzionamento del modulo USB_MIDI implementato come un processo che lavora in background. Come si può notare il dispositivo è sempre in stato di ricezione aspettando che sul canale venga inviato un messaggio. Una volta che esso viene ricevuto, viene generato un interrupt e un modulo si occupa di decodificare il messaggio e di chiamare delle procedure che si occupano di modificare i parametri del sintetizzatore.

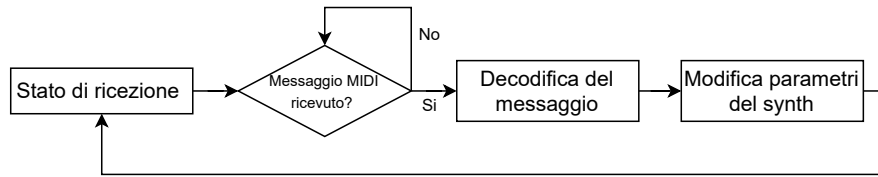


Figura 10: Funzionamento della classe MIDI-USB.

2.5 Gestione del DAC

Su STM32F4 ci sono 3 periferiche che gestiscono il protocollo I2C e 2 periferiche che gestiscono il protocollo I2S. In questo progetto sono state scelte le periferiche I2C1 e I2S3 perchè sono direttamente collegate al convertitore presente sulla board. Durante la fase di inizializzazione, per il protocollo I2C, è stata impostata una velocità di clock di 100 kHz (standard mode) e una modalità di indirizzamento di 7 bit. Per quanto riguarda il protocollo I2S è stata impostata la modalità 'master transmit', lo standard adottato è quello della Philips (lo stesso vale per il DAC), la frequenza di campionamento è di 48kHz, i campioni sono rappresentati da dati su 16 bit in pacchetti da 16 bit e come sorgente del clock è stato impostato l'oscillatore esterno. Tramite il protocollo I2C sono stati inizializzati anche i parametri del DAC come: il gain unitario, la frequenza di campionamento di 48 kHz e la trasmissione dei dati tramite il protocollo I2S usando pacchetti di 16 bit.

Terminata la fase di inizializzazione viene avviato il protocollo I2S in modalità DMA. Esso si occuperà di inviare serialmente i campioni audio computati dal microcontrollore senza l'uso della CPU. Per la computazione e il salvataggio dei dati è stata utilizzata una tecnica chiamata double buffering. Essa consiste nel dividere il buffer in cui salvo i campioni in 2 parti uguali. Mentre il microcontrollore si occupa di computare i campioni da salvare nella prima metà, il DMA si occupa di inviare la seconda metà del buffer al convertitore. Una volta terminato l'invio della seconda metà viene generato un interrupt e chiamata la callback *HAL_I2S_TxCpltCallback* per notificare la CPU di iniziare l'invio della prima metà e riempire la seconda metà del buffer con nuovi campioni. Una volta terminato l'invio della prima metà viene generato un interrupt e chiamata la callback *HAL_I2S_TxHalfCpltCallback* e il processo riparte da capo. Il tutto è riassunto nel diagramma 11.

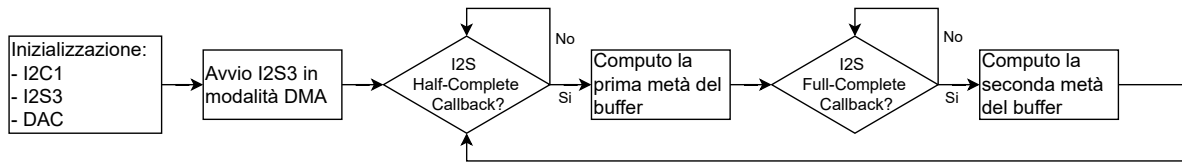


Figura 11: Funzionamento del DAC e I2S.

2.6 Gestione dell'algoritmo del synth

Come si può notare dalla figura 11 la generazione del suono è una task svolta a seguito di un interrupt scaturito dal DMA. Al verificarsi di questo evento, viene chiamata la callback I2S nella quale viene avviato l'algoritmo di generazione del suono che ciclicamente ha il compito di riempire un buffer campione per campione come mostrato nella figura 12. Terminato il ciclo il controllo è restituito alla parte di programma a bassa priorità. Al verificarsi dell'interrupt successivo, il buffer riempito nella callback precedente viene inviato al DAC tramite il protocollo I2S, mentre viene riempito il nuovo buffer con lo stesso procedimento.

Ogni blocco dell'algoritmo in figura 12 riceve in input dei parametri comandati da switch (gestiti tramite interrupt), convertitori analogico-digitali (gestiti tramite DMA) e USB_MIDI (gestiti tramite interrupt e polling). In figura 13 sono mostrati i parametri che vengono gestiti da ogni blocco e quale periferica si occupa di loro.

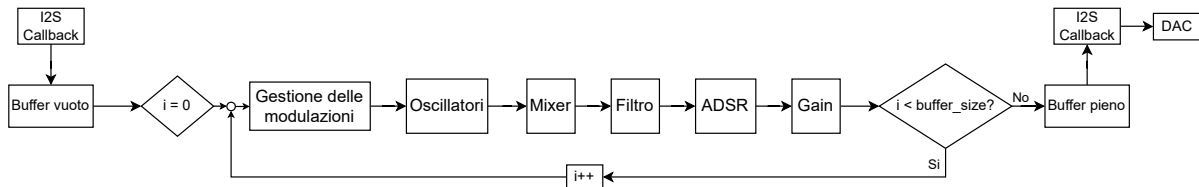


Figura 12: Diagramma di flusso del DSP.

3 Componenti Hardware e Circuiteria

Componenti interni al MikroMOOD:

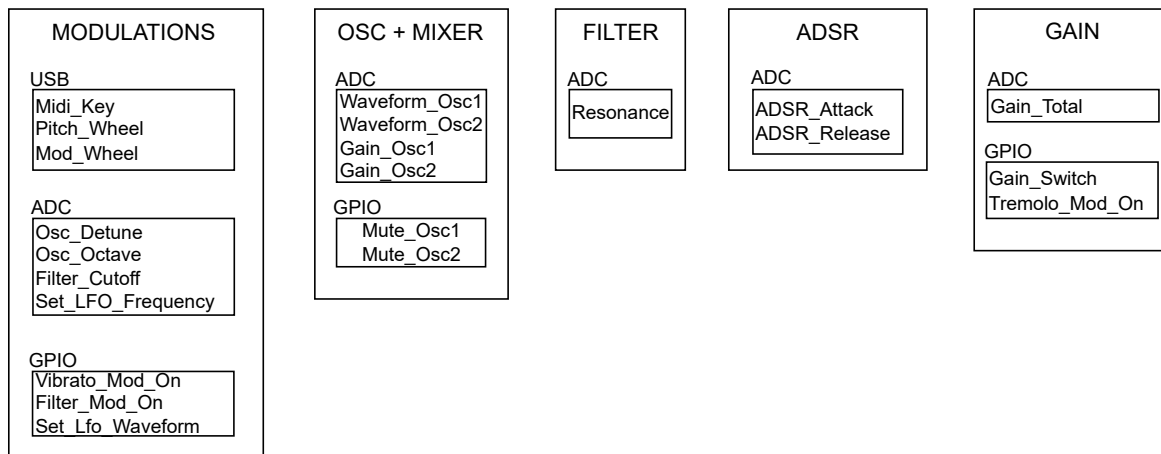


Figura 13: I/O per ogni blocco.

- Scheda STM32F407G - DISCOVERY²: è la board di ST sulla quale verrà sviluppato il prototipo. Il microcontrollore è basato su architettura ARM con un processore a 32-bit Cortex-M4 con un clock fino a 168 MHz, 1 Mbyte di memoria Flash e 196 Kbyte di SRAM. La capacità computazionale e di memoria fornita dal microcontrollore va ben oltre quella necessaria e potevano essere fatte scelte più economiche e meno potenti ma, i principali motivi per cui è stata scelta questa board sono:
 - Disponibilità di un connettore USB On-The-Go che permette di ricevere dati dalla tastiera MIDI assumendo il ruolo di Host nella comunicazione USB.
 - Disponibilità di un convertitore D/A Sigma-Delta con una risoluzione nominale di 24 bit³. In questo progetto la qualità dell'audio è fondamentale e la risoluzione di 12 bit dei DAC presenti nel microcontrollore non basta, ecco perchè si necessita di un convertitore esterno e in questo caso già presente sulla board.
 - Disponibilità di un jack femmina da 3.5mm saldato sulla scheda e direttamente connesso al DAC al quale si può collegare l'altoparlante in uscita.
- Componentistica per il pannello di controllo: 7 switch On/Off, 2 switch multipli per selezionare le forme d'onda degli oscillatori e 11 potenziometri per il controllo degli altri parametri.
- Componentistica per il collaudo e i test: almeno 3 breadboard, cavetti maschio-femmina, eventuali adattatori per l'ingresso USB e il jack in uscita.

²Scheda STM32F407G: <https://www.st.com/en/evaluation-tools/stm32f4discovery.html>

³Il convertitore montato sulla scheda è il CS43L22: <https://www.cirrus.com/products/cs43l22/>

3.1 Circuiteria degli input

Come mostra la figura 14 ci sono 2 tipi di circuiti in input: quello per il collegamento degli switch e quello per il collegamento dei potenziometri. Nel primo caso, in base alla posizione dell'interruttore, il pin legge una tensione di 3.3V oppure di 0V. Gli switch servono per attivare o disattivare alcune funzionalità del sintetizzatore come le modulazioni o gli oscillatori. Nel secondo caso, in base alla posizione del potenziometro, sul pin del microcontrollore viene letta una tensione analogica variabile tra 0V e 3.3V. Essi servono per modificare parametri come la frequenza degli oscillatori, il volume, i parametri del filtro e dell'adsr.

3.2 Circuiteria degli output

La figura 15 mostra i collegamenti circuitali già presenti sulla board per far funzionare correttamente il DAC. Come si può notare i pin 1 (SDA) e 2 (SCL) sono quelli utilizzati per la comunicazione I2C e sono direttamente collegati ai pin PB6 e PB9 del micro (I2C1). I pin 37 (MCLK), 38 (SCLK), 39 (SDIN) e 40 (LRCK) sono utilizzati per la comunicazione I2S e sono direttamente collegati ai pin PC7, PC10, PC12, PA4 del micro (IS3). Infine il pin 32 (RST) è direttamente collegato al pin PD4 del micro. Il compito del programmatore è quello di impostare correttamente i pin del microcontrollore per farli lavorare con il DAC secondo i protocolli indicati.

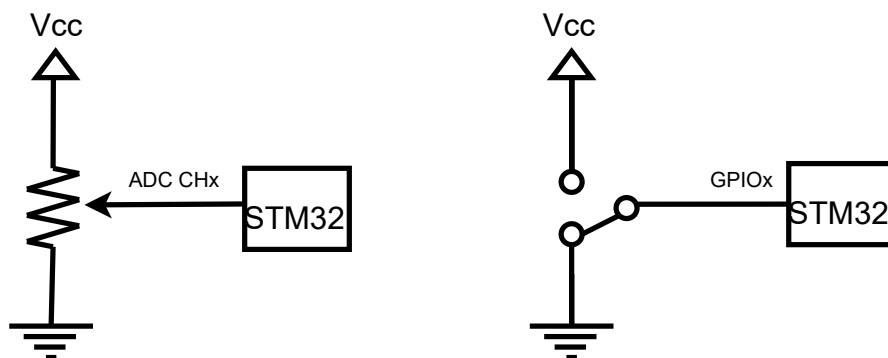


Figura 14: Circuito di collegamento degli switch (a sinistra) e del potenziometro (a destra).

del tempo che il DMA impiega per trasferire tutti i dati. Per esempio se la dimensione del buffer è 512, il tempo che impiega il microcontrollore per riempire il buffer è di 4 ms, mentre il tempo che intercorre tra un DMA interrupt e il successivo è 10,66 ms. Questo vuol dire che indicativamente il microcontrollore ha $10,66\text{ms} - 4\text{ms} = 6,66\text{ms}$ per eseguire altre funzionalità come in questo caso gestire il protocollo USB.

5 Analisi di Potenza Dissipata

In questo capitolo, analizziamo la potenza dissipata dai vari componenti hardware utilizzati nel progetto. Gli elementi del progetto da considerare sono i seguenti:

- STM32F407: $3.3V * 55.95mA = 184.6mW$
- CS43L22 (DAC): $25.48mW$
- 11 Potenzimetri: $11 * (3.3V * 0.66mA) = 2.4mW$
- Potenza totale: $184.6mW + 25.48mW + 2.4mW = 212.48mW$

La potenza del convertitore è stata reperita dalla sezione 'Power Consumption' del datasheet (pag. 20). La potenza del microcontrollore è stata calcolata usando il tool fornito da STM32CubeIDE. Essa tiene conto sia della potenza dissipata dal chip stesso che dall'uso delle periferiche, ovvero: ADC1, GPIOA/B/C/D/E/H, I2C1, I2S3, TIM2 e USB_OTG_FS.

6 Conclusione

In conclusione, si può dire che gli obiettivi prefissati nel paragrafo 1.1 sono stati tutti soddisfatti:

- La gestione degli input viene eseguita correttamente. Il cambio di stato degli switch e dei potenziometri viene letto prontamente dal microcontrollore e i parametri vengono a loro volta modificati.
- Il driver scritto appositamente per il DAC CS43L22 funziona correttamente, sia per quanto riguarda la fase di inizializzazione tramite il protocollo I2C e quella di scambio dei dati tramite il protocollo I2S.

- Il protocollo MIDI_USB funziona correttamente. Grazie alla classe scritta è possibile riconoscere la pressione dei tasti e lo spostamento delle manopole 'Pitch Wheel' e 'Modulation Wheel' della tastiera MIDI.
- Gli algoritmi di generazione del suono funzionano correttamente.
- Il circuito montato su breadboard funziona anche se scomodo da usare.

Di seguito sono elencati alcune migliorie o sviluppi futuri che possono essere effettuati:

- Spostare il codice di generazione del suono dalla callback DMA (ad alta priorità) nel ciclo while (a bassa priorità). In questo modo il programma sarebbe più "corretto" dal momento che le computazioni non si effettuano nelle routine di risposta all'interrupt nelle quali solitamente si cambiano solo dei parametri.
- Estendere le funzionalità MIDI. Il modulo sviluppato permette solo di leggere i tasti e le manopole 'Pitch Wheel' e 'Modulation Wheel'. Nel protocollo sono presenti altre funzionalità che non sono state implementate.
- Permettere l'interfacciamento tra il synth e tutte le tastiere MIDI. Il modulo sviluppato permette l'interfacciamento di alcune delle tastiere in commercio. Sono state provate alcune tastiere più 'vecchie' che non hanno il connettore USB ma solo quello MIDI. Usando un adattatore MIDI-USB è stato possibile connetterle al microcontrollore ma i descrittori del protocollo USB e le configurazioni cambiavano impossibilitando la trasmissione dei messaggi.
- Il sintetizzatore potrebbe essere esteso aggiungendo delle funzionalità o degli effetti per far sì che il prototipo sviluppato assomigli ancora di più all'originale.
- Il circuito potrebbe essere saldato e potrebbe essere creata una struttura in legno per contenere tutta la circuiteria e il pannello di controllo. In questo modo non solo il prodotto è più bello da vedere ma anche l'usabilità migliora.